

School Of Computing and Informatics

Systems Programming

Matrix Manipulation

Submitted By: Mohammad Aldeghemat

Summer – 2026

Table of Contents

Table of Contents 2

1. Systems Programming, Multiprocessing, and Multithreading ..... **Error! Bookmark not defined.**

1.1 Systems Programming ..... **Error! Bookmark not defined.**

1.2 Multiprocessing ..... **Error! Bookmark not defined.**

1.3 Multithreading ..... **Error! Bookmark not defined.**

2. Sequential Implementation 3

2.1 Functions 3

2.2 Main 4

3. Implementing multi-process 8

4. Implementing the IPC 11

4.1 Multiplication Process 13

4.2 Transpose Process 15

4.3 Average Process 16

4.4 Application Time 18

4.5 Communication Abilities Diagram 18

5. Implementing Multithreading Instead of Multi-processes 19

5.1 Main Threads 19

5.2 Multiplication Thread 20

5.3 Transpose Thread 20

5.4 Average Thread & Mutex 21

5.5 Thread Management Techniques ..... **Error! Bookmark not defined.**

Semaphore ..... **Error! Bookmark not defined.**

Thread Pool ..... **Error! Bookmark not defined.**

Condition Variable ..... **Error! Bookmark not defined.**

5.6 Race Condition in My Implementation 22

6. Client-Server Implementation 23

7. C vs Shell Scripting 25

7.1 Code Examples 26

8 Implementing Shell Scripting 27

References 29

1. Sequential Implementation

1.1 Functions

```
int **createTheMatrix(int r, int c){
   int **matrix = malloc(r * sizeof(int *));
   for(int i = 0; i < r; i++){
      matrix[i] = malloc(c * sizeof(int));
      }
   return matrix;
   }

void freeMatrix(int **matrix, int r){
   for(int i = 0; i < r; i++){
      free(matrix[i]);
   }
   free(matrix);
}
```

The first function is createTheMatrix(). It receives rows and columns. The first malloc() creates a pointer for every row, then the loop creates the columns inside each row. So, matrix[i] points to one row and the values inside that row can be accessed using matrix[i][j]. I used dynamic memory here because the dimensions are not fixed and they are generated when the program starts, and because I wanted to create a good setup to use later in the project. The freeMatrix() function free this memory when the matrix is not needed anymore, as the loop frees the rows one by one because they were created by separate calls to malloc(), and freeing one row also frees all the columns inside that row, then we free the main matrix pointer that was holding their addresses.

```
int **multiplication(int rA, int cA, int rB, int cB, int **matrixA, int **matrixB){
   if(cA != rB){
   return NULL;
   }
   int **res = createTheMatrix(rA, cB);
   for(int i = 0; i < rA; i++){
     for(int j = 0; j < cB; j++){
         res[i][j] = 0;
         for(int k = 0; k < cA; k++){
       res[i][j] += matrixA[i][k] * matrixB[k][j];
         }
      }
   }
   return res;
}
```

For multiplication, Matrix A and Matrix B are sent to multiplication() with their dimensions, then it compares cA with rB because these values need to be equal. If they are different, the function returns NULL. When they are equal, res is created using rA and cB, since the multiplication result takes its rows from Matrix A and its columns from Matrix B. The first loop chooses a row from Matrix A and the second loop chooses a column from Matrix B, and the value of res[i][j] starts from zero, then the third loop goes through the selected row and column, multiplies their values and adds them to the result. When the loops finish, the function return res to the main().

```
int **transpose(int r, int c, int **matrix){
   int **res = createTheMatrix(c, r);
   for(int i = 0; i < r; i++){
       for(int j = 0; j < c; j++){
          res[j][i] = matrix[i][j];
       }
   }
   return res;
}
```

For the transpose, I created another matrix instead of changing the original matrix, so the original matrix stays the same for other operations. I passed c as its rows and r as its columns because the two dimensions changes their places after transposing and the loops goes through the original matrix, res[j][i] = matrix[i][j] moves each value to its new position and res is returned after that to the main().

```
double average(int r, int c, int **matrix){
   int sum = 0;
   int count = r * c;
   double avg = 0;
   for(int i = 0; i < r; i++){
      for(int j = 0; j < c; j++){
         sum += matrix[i][j];
      }
   }
   avg = (double) sum / count;
   return avg;
}
```

For the average, sum starts from zero and the two loops go through the rows and columns of the selected matrix. Every visited value is added to sum, then I divided it by r * c because this gives the total number of values inside the matrix, then the function returns the average.

1.2 Main

```
int main(){
srand(time(NULL));
```

```
clock_t start;
clock_t end;
clock_t overallStart;
clock_t overallEnd;
double multTime;
double transposeTime;
double avgTime;
double overallTime;
overallStart = clock();
int rA = rand() % 10 + 2;
int cA = rand() % 10 + 2;
int rB = cA;
int cB = rand() % 10 + 2;
int **matrixA = createTheMatrix(rA, cA);
int **matrixB = createTheMatrix(rB, cB);
int **multRes;
int **transRes;
double avgResult;
char choiceOne;
char choiceTwo;
for(int i = 0; i < rA; i++){
   for(int j = 0; j < cA; j++){
      matrixA[i][j] = rand() % 9 + 1;
   }
}
printf("================================================================\n");
printf("Matrix A (%d X %d)\n", rA, cA);
for(int i = 0; i < rA; i++){
   for(int j = 0; j < cA; j++){
      printf("%d ", matrixA[i][j]);
   }
   printf("\n");
}
for(int i = 0; i < rB; i++){
   for(int j = 0; j < cB; j++){
      matrixB[i][j] = rand() % 9 + 1;
   }
}
printf("================================================================\n");
printf("Matrix B (%d X %d)\n", rB, cB);
for(int i = 0; i < rB; i++){
   for(int j = 0; j < cB; j++){
   printf("%d ", matrixB[i][j]);
   }
   printf("\n");
```

```
}
```

In the first part of main(), I called srand(time(NULL)) before generating anything so the dimensions and values can change each time we run the application. I also created the variables required for measuring the time of every operation and the overall application time. The dimensions of Matrix A are stored in rA and cA, and both of them can have a value from 2 to 11. I made rB equal to cA because these two values need to match for multiplication, and cB also can have a value from 2 to 11 which are the dimensions of MatrixB. After deciding the dimensions, I created Matrix A and Matrix B by calling createTheMatrix() function. I also declared multRes, transRes and avgResult to store the returned results later, while choiceOne and choiceTwo will store the user choices in which matrix to transpose or calculate the average to.

After that, I used two loops to fill each matrix with random values from 1 to 9, then another two loops are used to print its values. I also printed the dimensions and the values inside each matrix.

```
printf("================================================================\n");
printf("Which Matrix Do You Want To Transpose? (A/B)\n");
scanf(" %c", &choiceOne);
printf("================================================================\n");
printf("Which Matrix Do You Want To Calculate Average For? (A/B)\n");
scanf(" %c", &choiceTwo);
start = clock();
multRes = multiplication(rA, cA, rB, cB, matrixA, matrixB);
end = clock();
multTime = (double) (end - start) / CLOCKS_PER_SEC;
printf("================================================================\n");
printf("Multiplication Result:\n");
for (int i = 0; i < rA; i++){
    for (int j = 0; j < cB; j++){
        printf("%d ", multRes[i][j]);
    }
    printf("\n");
}
start = clock();
if(choiceOne == 'A' || choiceOne == 'a'){
   transRes = transpose(rA, cA, matrixA);
}else if(choiceOne == 'B' || choiceOne == 'b'){
   transRes = transpose(rB, cB, matrixB);
}else{
  printf("Invalid Choice\n");
  return 1;
}
end = clock();
transposeTime = (double) (end - start) / CLOCKS_PER_SEC;
```

```
printf("============================================================\n");
if(choiceOne == 'A' || choiceOne == 'a'){
   printf("Transpose of Matrix A:\n");
   for (int i = 0; i < cA; i++){
       for (int j = 0; j < rA; j++){
           printf("%d ", transRes[i][j]);
       }
       printf("\n");
    }
}else if(choiceOne == 'B' || choiceOne == 'b'){
   printf("Transpose of Matrix B:\n");
   for(int i = 0; i < cB; i++){
      for(int j = 0; j < rB; j++){
         printf("%d ", transRes[i][j]);
       }
       printf("\n");
    }
}
start = clock();
if(choiceTwo == 'A' || choiceTwo == 'a'){
   avgResult = average(rA, cA, matrixA);
}else if(choiceTwo == 'B' || choiceTwo == 'b'){
   avgResult = average(rB, cB, matrixB);
}else{
  printf("Invalid Choice\n");
  return 1;
}
end = clock();
avgTime = (double)(end - start) / CLOCKS_PER_SEC;
printf("============================================================\n");
if(choiceTwo == 'A' || choiceTwo == 'a'){
   printf("Average of Matrix A is: %f\n", avgResult);
}else if(choiceTwo == 'B' || choiceTwo == 'b'){
   printf("Average of Matrix B is: %f\n", avgResult);
}else{
  printf("Invalid Choice\n");
  return 1;
}
overallEnd = clock();
overallTime = (double)(overallEnd - overallStart) / CLOCKS_PER_SEC;
printf("============================================================\n");
printf("Execution Times:\n");
printf("Multiplication: %f seconds\n", multTime);
printf("Transpose: %f seconds\n", transposeTime);
printf("Average: %f seconds\n", avgTime);
```

```
printf("Application: %f seconds\n", overallTime);
freeMatrix(matrixA, rA);
freeMatrix(matrixB, rB);
freeMatrix(multRes, rA);
if (choiceOne == 'A' || choiceOne == 'a'){
    freeMatrix(transRes, cA);
}
else{
    freeMatrix(transRes, cB);
}
return 0;
}
```

After printing Matrix B, I asked the user two times to take his choice of which matrix should be transposed and which matrix we should calculate the average for. Before calling each function, I saved the clock value in start variable and after each function I saved the clock value in end variable to calculate the time for each function.

Matrix A, Matrix B and their dimensions are then passed to the multiplication() function, while the pointer returned from it is stored in multRes, then the result is printed.

After that, choiceOne variable is checked to know which matrix should be passed to the transpose() function. If the user selects Matrix A, its matrix pointer and dimensions are passed, and the same thing is done for Matrix B if it was selected. The pointer returned from the function is stored in transRes, then the transpose result is printed.

The same thing is done for the average using choiceTwo variable, where Matrix A or Matrix B is passed to the average() function, and the value returned from it is stored in avgResult, then it is printed depending on the matrix that was selected. The functions are called in the order of multiplication, transpose and average, and each one is completed before the next one starts, which makes the solution execute sequentially.

At the end, we print execution time of the whole application and all matrix operations including multiplication, transpose, and average. Also, freeMatrix() is called for Matrix A, Matrix B the multiplication result, and the transpose result.

2. Implementing multi-process

In the multi-process implementation, the logic remains the same, but instead of executing the multiplication, transpose and average functions sequentially, I created three child processes, where each one executes one function, prints the result and calculates the time taken. A point worth mentioning is that because the operations run concurrently, we will consider the longest time taken as the time required for the three processes to finish. Also, I removed the part where I calculate the application time because clock() in the parent process does not include the time used by the child processes. Later, when pipes

are used, I will add the application time again, send the execution time from each child process to the parent, determine the longest time and add it to the application time.

```
pid_t firstChild;
pid_t secondChild;
pid_t thirdChild;
```

First, I declared three variables of type pid_t, where each variable is used to store the returned fork() value.

```
firstChild = fork();
if(firstChild == 0){
start = clock();
multRes = multiplication(rA, cA, rB, cB, matrixA, matrixB);
end = clock();
multTime = (double) (end - start) / CLOCKS_PER_SEC;
printf("================================================================\n");
printf("Multiplication Result:\n");

for (int i = 0; i < rA; i++){
   for (int j = 0; j < cB; j++){
       printf("%d ", multRes[i][j]);
    }
    printf("\n");
}
printf("================================================================\n");
printf("Multiplication Execution Time: %f seconds\n", multTime);
freeMatrix(matrixA, rA);
freeMatrix(matrixB, rB);
freeMatrix(multRes, rA);
exit(0);
}

secondChild = fork();
if(secondChild == 0){
start = clock();
if(choiceOne == 'A' || choiceOne == 'a'){
   transRes = transpose(rA, cA, matrixA);
}else if(choiceOne == 'B' || choiceOne == 'b'){
   transRes = transpose(rB, cB, matrixB);
}else{
  printf("Invalid Choice\n");
  return 1;
}
end = clock();
transposeTime = (double) (end - start) / CLOCKS_PER_SEC;
printf("================================================================\n");
```

```
if(choiceOne == 'A' || choiceOne == 'a'){
   printf("Transpose of Matrix A:\n");
   for (int i = 0; i < cA; i++){
       for (int j = 0; j < rA; j++){
           printf("%d ", transRes[i][j]);
       }
       printf("\n");
    }
}else if(choiceOne == 'B' || choiceOne == 'b'){
   printf("Transpose of Matrix B:\n");
   for(int i = 0; i < cB; i++){
      for(int j = 0; j < rB; j++){
         printf("%d ", transRes[i][j]);
       }
       printf("\n");
    }
}
printf("============================================================\n");
printf("Transpose Execution Time: %f seconds\n", transposeTime);
freeMatrix(matrixA, rA);
freeMatrix(matrixB, rB);
if (choiceOne == 'A' || choiceOne == 'a'){
    freeMatrix(transRes, cA);
}
else{
    freeMatrix(transRes, cB);
}
exit(0);
}

thirdChild = fork();
if(thirdChild == 0){
start = clock();
if(choiceTwo == 'A' || choiceTwo == 'a'){
   avgResult = average(rA, cA, matrixA);
}else if(choiceTwo == 'B' || choiceTwo == 'b'){
   avgResult = average(rB, cB, matrixB);
}else{
  printf("Invalid Choice\n");
  return 1;
}
end = clock();
avgTime = (double)(end - start) / CLOCKS_PER_SEC;
printf("============================================================\n");
```

```
if(choiceTwo == 'A' || choiceTwo == 'a'){
   printf("Average of Matrix A is: %f\n", avgResult);
}else if(choiceTwo == 'B' || choiceTwo == 'b'){
   printf("Average of Matrix B is: %f\n", avgResult);
}else{
  printf("Invalid Choice\n");
  return 1;
}
printf("============================================================\n");
printf("Average Execution Time: %f seconds\n", avgTime);
freeMatrix(matrixA, rA);
freeMatrix(matrixB, rB);
exit(0);
}

wait(NULL);
wait(NULL);
wait(NULL);
freeMatrix(matrixA, rA);
freeMatrix(matrixB, rB);
return 0;
}
```

The first child process executes the multiplication function, the second child executes the transpose function, and the third child executes the average function. Each child also calculates and prints the execution time of its own operation. I used exit(0) at the end of each child process so that after the child finishes its assigned operation, it terminates and does not continue to the next fork() statements and create additional processes. After creating the three child processes, I used three wait(NULL) statements. They are placed after all three processes are created, which allows the three operations to run concurrently first, while the parent then waits for all of them to finish before ending the program.

3. Implementing the IPC

In the IPC implementation, the three main processes are still used for multiplication, transpose and average, but now I used pipes to transfer the required data and results between the processes because every process has its own separate address space. I also divided the multiplication and average operations into smaller subprocesses, where every subprocess calculates one part of the operation and sends its result back to its main process. Separate pipes are also used to send the execution times, while a named pipe is used to send the multiplication result directly from the multiplication process to the transpose process.

```
int parentToMult[2];
int multToParent[2];
```

```
int parentToAvg[2];
int avgToParent[2];
int parentToTrans[2];
int transToParent[2];
int multTimePipe[2];
int transTimePipe[2];
int avgTimePipe[2];
pipe(parentToMult);
pipe(multToParent);
pipe(parentToAvg);
pipe(avgToParent);
pipe(parentToTrans);
pipe(transToParent);
pipe(multTimePipe);
pipe(transTimePipe);
pipe(avgTimePipe);
```

First, I created two pipes for every main operation. parentToMult, parentToTrans and parentToAvg are used by the parent to send the required data to each child process, while multToParent, transToParent and avgToParent are used by the child processes to return their results to the parent. I also created multTimePipe, transTimePipe and avgTimePipe as separate pipes that are only used to send the execution time of every main process to the parent.

```
firstChild = fork();
if(firstChild == 0){
   ...
   exit(0);
}
secondChild = fork();
if(secondChild == 0){
   ...
   exit(0);
}
thirdChild = fork();
if(thirdChild == 0){
   ...
   exit(0);
}
int rA = rand() % 10 + 2;
int cA = rand() % 10 + 2;
int rB = cA;
int cB = rand() % 10 + 2;
int **matrixA;
int **matrixB;
```

The three child processes are created before Matrix A and Matrix B are created in the parent. So, unlike the previous multi-process implementation, the children do not already have the matrices, and they need to receive the dimensions and values using pipes.

3.1 Multiplication Process

```
write(parentToMult[1], &rA, sizeof(int));
write(parentToMult[1], &cA, sizeof(int));
write(parentToMult[1], &rB, sizeof(int));
write(parentToMult[1], &cB, sizeof(int));
for(int i = 0; i < rA; i++){
   write(parentToMult[1], matrixA[i], cA * sizeof(int));
}
for(int i = 0; i < rB; i++){
   write(parentToMult[1], matrixB[i], cB * sizeof(int));
}
/* The selected matrix is sent to transpose and average */
```

For multiplication, I first send the four dimensions because the multiplication process needs both Matrix A and Matrix B, then I send both matrices row by row. I send them row by row because every row was created by a separate call to malloc(). For transpose and average, only the matrix selected by the user is sent. I send the required data to all three processes before receiving the results, so the three operations can start their work concurrently.

```
int processesNum;
if(rA <= 10){
   processesNum = rA;
}else if(rA <= 100){
   processesNum = 10;
}else if(rA <= 1000){
   processesNum = 20;
}else{
   processesNum = 100;
}
int rowsForProcess = rA / processesNum;
int extraRows = rA % processesNum;
int current = 0;
int childToMult[processesNum][2];
int multSubTimePipe[2];
pipe(multSubTimePipe);
for(int i = 0; i < processesNum; i++){
   pipe(childToMult[i]);
}
```

The multiplication process receives Matrix A and Matrix B from the parent, then I decide the number of subprocesses depending on the number of rows in Matrix A.

rowsForProcess stores the normal number of rows for each subprocess, while extraRows contains any rows left after the division.

```
for(int i = 0; i < processesNum; i++){
   int rowsNow = rowsForProcess;
   if(extraRows > 0){
      rowsNow++;
      extraRows--;
   }
   multChild = fork();
   if(multChild == 0){
      start = clock();
      int **childRes = createTheMatrix(rowsNow, cB);
      for(int j = 0; j < rowsNow; j++){
         for(int k = 0; k < cB; k++){

            childRes[j][k] = 0;

            for(int x = 0; x < cA; x++){
               childRes[j][k] += matrixA[current + j][x] * matrixB[x][k];
            }
         }
      }
      write(childToMult[i][1], &current, sizeof(int));
      write(childToMult[i][1], &rowsNow, sizeof(int));
      for(int j = 0; j < rowsNow; j++){
         write(childToMult[i][1], childRes[j], cB * sizeof(int));
      }
      freeMatrix(childRes, rowsNow);
      end = clock();
      time = (double)(end - start) / CLOCKS_PER_SEC;
      write(multSubTimePipe[1], &time, sizeof(double));
      exit(0);
   }
   current += rowsNow;
}
```

Each subprocess calculates its assigned rows only. current is used to know where the subprocess starts, while rowsNow contains the number of rows it should calculate. After finishing, the subprocess sends its calculated rows back to the multiplication process and sends its execution time separately using multSubTimePipe.

```
int **multRes = createTheMatrix(rA, cB);
for(int i = 0; i < processesNum; i++){
   int start;
   int rows;
```

```
   read(childToMult[i][0], &start, sizeof(int));
   read(childToMult[i][0], &rows, sizeof(int));
   for(int j = start; j < start + rows; j++){
      read(childToMult[i][0], multRes[j], cB * sizeof(int));
   }
}
for(int i = 0; i < processesNum; i++){
   double time;
   read(multSubTimePipe[0], &time, sizeof(double));
   if(time > multTime){
      multTime = time;
   }
}
```

The multiplication process receives the parts calculated by the subprocesses and places them inside multRes. I also receive their execution times and keep the longest one because the subprocesses run concurrently.

```
for(int i = 0; i < rA; i++){
   write(multToParent[1], multRes[i], cB * sizeof(int));
}
int namedPipeWrite;
namedPipeWrite = open("mult_transpose", O_WRONLY);
write(namedPipeWrite, &rA, sizeof(int));
write(namedPipeWrite, &cB, sizeof(int));
for(int i = 0; i < rA; i++){
   write(namedPipeWrite, multRes[i], cB * sizeof(int));
}
close(namedPipeWrite);
...
multTime += (double)(end - start) / CLOCKS_PER_SEC;
write(multTimePipe[1], &multTime, sizeof(double));
```

The complete multiplication result is sent to the parent, and the same result is also sent directly to the transpose process using the named pipe mult_transpose. The longest subprocess time is added to the multiplication process time because clock() inside this process does not include the CPU time used by its subprocesses.

3.2 Transpose Process

```
read(parentToTrans[0], &rows, sizeof(int));
read(parentToTrans[0], &cols, sizeof(int));
int **transMatrix = createTheMatrix(rows, cols);
for(int i = 0; i < rows; i++){
   read(parentToTrans[0], transMatrix[i], cols * sizeof(int));
}
int **transRes = transpose(rows, cols, transMatrix);
```

```
for(int i = 0; i < cols; i++){
   write(transToParent[1], transRes[i], rows * sizeof(int));
}
```

The transpose process receives the matrix selected by the user, uses the same transpose() function from the previous implementations and sends the result back to the parent.

```
int namedPipeRead;
namedPipeRead = open("mult_transpose", O_RDONLY);
read(namedPipeRead, &multRows, sizeof(int));
read(namedPipeRead, &multCols, sizeof(int));
int **multMatrix = createTheMatrix(multRows, multCols);
for(int i = 0; i < multRows; i++){
   read(namedPipeRead, multMatrix[i], multCols * sizeof(int));
}
close(namedPipeRead);
int **multTransRes = transpose(multRows, multCols, multMatrix);
multTransRes = realloc(multTransRes, (multCols + 1) * sizeof(int *));
multTransRes[multCols] = malloc(multRows * sizeof(int));
for(int i = 0; i < multRows; i++){
   multTransRes[multCols][i] = rand() % 9 + 1;
}
```

The transpose process also receives the multiplication result directly through mult_transpose. After receiving the matrix, I transpose it and use realloc() to add one new row. I then allocate this row using malloc() and fill it with random numbers from 1 to 9.

```
for(int i = 0; i < multCols + 1; i++){
   write(transToParent[1], multTransRes[i], multRows * sizeof(int));
}
...
transposeTime = (double)(end - start) / CLOCKS_PER_SEC;
write(transTimePipe[1], &transposeTime, sizeof(double));
```

The final matrix is sent back to the parent, then the transpose process calculates its execution time and sends it using transTimePipe.

3.3 Average Process

```
int rowsForProcess = rows / processesNum;
int extraRows = rows % processesNum;
int current = 0;
for(int i = 0; i < processesNum; i++){
   int rowsNow = rowsForProcess;
   if(extraRows > 0){
      rowsNow++;
      extraRows--;
   }
   avgChild = fork();
```

```
    if(avgChild == 0){
        start = clock();
        int sum = 0;
        for(int j = current; j < current + rowsNow; j++){
            for(int k = 0; k < cols; k++){
                sum += avgMatrix[j][k];
            }
        }
        write(childToAvg[i][1], &sum, sizeof(int));
        end = clock();
        time = (double)(end - start) / CLOCKS_PER_SEC;
        write(avgSubTimePipe[1], &time, sizeof(double));
        exit(0);
    }
    current += rowsNow;
}
```

The average process uses the same subprocess idea as multiplication. Every subprocess calculates the sum of its assigned rows and sends the partial sum back to the average process. The execution time of each subprocess is also sent separately.

```
int totalSum = 0;
for(int i = 0; i < processesNum; i++){
    int sum;
    read(childToAvg[i][0], &sum, sizeof(int));
    totalSum += sum;
}
for(int i = 0; i < processesNum; i++){
    double time;
    read(avgSubTimePipe[0], &time, sizeof(double));
    if(time > avgTime){
        avgTime = time;
    }
}
double avgResult = (double) totalSum / (rows * cols);
write(avgToParent[1], &avgResult, sizeof(double));
...
avgTime += (double)(end - start) / CLOCKS_PER_SEC;
write(avgTimePipe[1], &avgTime, sizeof(double));
```

The partial sums are added together inside totalSum, then I divide it by rows cols to calculate the final average. I also keep the longest subprocess time instead of adding all of them because the subprocesses run concurrently. The final average and execution time are then sent back to the parent.

3.4 Application Time

```
read(multTimePipe[0], &multTime, sizeof(double));
read(transTimePipe[0], &transposeTime, sizeof(double));
read(avgTimePipe[0], &avgTime, sizeof(double));
...
clock_t overallEnd = clock();
applicationTime = (double)(overallEnd - overallStart) / CLOCKS_PER_SEC;
if(multTime >= transposeTime && multTime >= avgTime){
   applicationTime += multTime;
}else if(transposeTime >= multTime && transposeTime >= avgTime){
   applicationTime += transposeTime;
}else{
   applicationTime += avgTime;
}
```

The partial sums are added together inside totalSum, then I divide it by rows cols to calculate the final average. I also keep the longest subprocess time instead of adding all of them because the subprocesses run concurrently. The final average and execution time are then sent back to the parent.

3.5 Communication Abilities Diagram

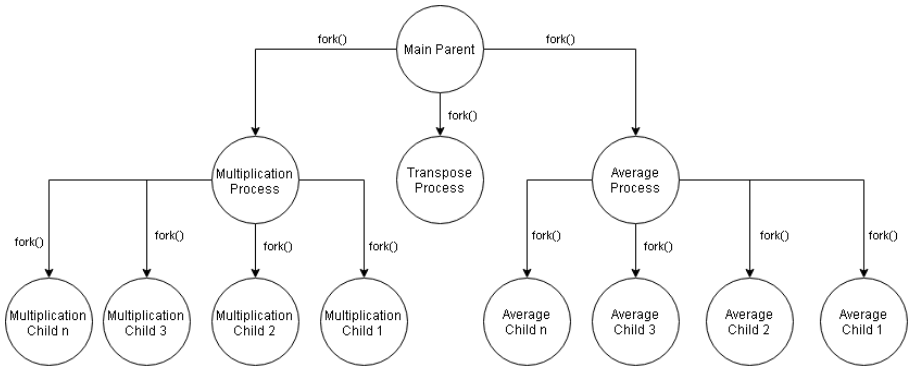


Figure 1: Structure

Figure 1 shows the process structure of the application. The main parent creates three processes for multiplication, transpose and average, while the multiplication and average processes create their own subprocesses to divide the work.

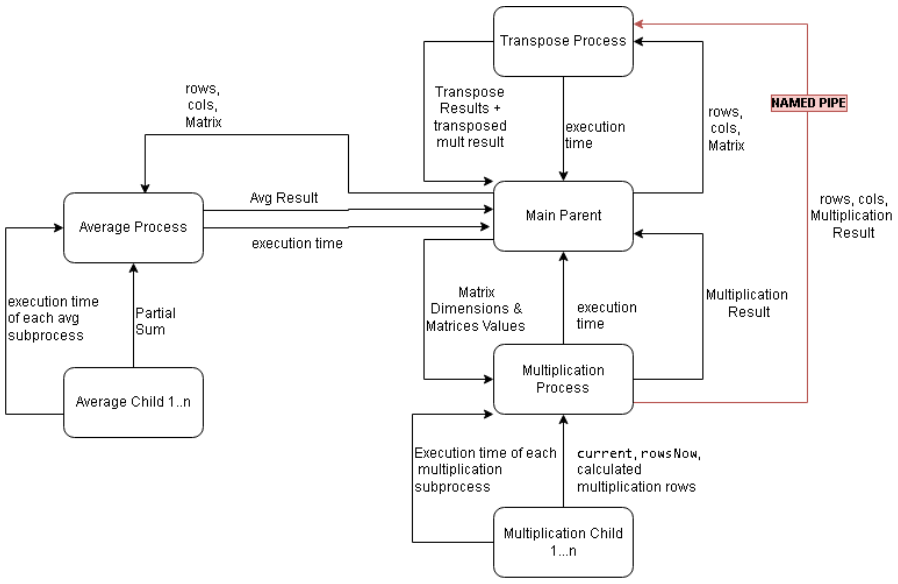


Figure 2: Communication Abilities

Figure 2 shows the communication between the processes using IPC. The parent sends the required matrices and dimensions, while the results and execution times are sent back. The multiplication process also sends its result directly to the transpose process using the mult_transpose named pipe.

4. Implementing Multithreading Instead of Multi-processes

For this implementation, I replaced the three main processes with three threads for multiplication, transpose and average. Since the threads are inside the same process, Matrix A, Matrix B and the results are shared between them, so I did not need to use pipes.

4.1 Main Threads

```
pthread_t firstThread;
pthread_t secondThread;
pthread_t thirdThread;
pthread_create(&firstThread, NULL, multiplicationThread, NULL);
pthread_create(&secondThread, NULL, transposeThread, NULL);
pthread_create(&thirdThread, NULL, averageThread, NULL);
pthread_join(firstThread, NULL);
pthread_join(secondThread, NULL);
pthread_join(thirdThread, NULL);
```

I created three threads, one for each operation, using pthread_create(). After that, I used pthread_join() for the three threads. I used join because the main thread should wait until multiplication, transpose and average finish before it prints the results and frees the matrices. Without join, the main thread could continue while one of the threads is still working.

4.2 Multiplication Thread

```
int rowsForThread = rA / threadsNum;
int extraRows = rA % threadsNum;
int current = 0;
pthread_t threads[threadsNum];
int threadData[threadsNum][2];
for(int i = 0; i < threadsNum; i++){
   int rowsNow = rowsForThread;
   if(extraRows > 0){
      rowsNow++;
      extraRows--;
   }
   threadData[i][0] = current;
   threadData[i][1] = rowsNow;
   pthread_create(&threads[i], NULL, multiplicationSubThread, threadData[i]);
   current += rowsNow;
}
```

The multiplication thread uses the same row division idea from the previous implementation. rowsForThread stores the normal number of rows for each subthread, while extraRows contains any rows left after the division. current is used to know where the subthread starts, while rowsNow contains the number of rows it should calculate.

```
for(int i = current; i < current + rowsNow; i++){
   for(int j = 0; j < cB; j++){
      multRes[i][j] = 0;
      for(int k = 0; k < cA; k++){
         multRes[i][j] += matrixA[i][k] * matrixB[k][j];
      }
   }
}
```

Each subthread calculates its assigned rows and stores them directly inside multRes. I did not use a mutex here because every subthread works on different rows, so they do not change the same values. After that, I used pthread_join() to wait until all the multiplication subthreads finish.

4.3 Transpose Thread

```
void *transposeThread(void *arg){
   if(choiceOne == 'A' || choiceOne == 'a'){
      transRes = transpose(rA, cA, matrixA);
   }else{
      transRes = transpose(rB, cB, matrixB);
   }
   return NULL;
}
```

The transpose thread checks the matrix selected by the user and uses the same transpose() function from the previous implementations. Since the matrices are shared between the threads, the transpose thread can use Matrix A or Matrix B directly and store the result inside transRes.

4.4 Average Thread & Mutex

```
int rowsForThread = rows / threadsNum;
int extraRows = rows % threadsNum;
int current = 0;
pthread_t threads[threadsNum];
int threadData[threadsNum][2];
for(int i = 0; i < threadsNum; i++){
   int rowsNow = rowsForThread;
   if(extraRows > 0){
      rowsNow++;
      extraRows--;
   }
   threadData[i][0] = current;
   threadData[i][1] = rowsNow;
   pthread_create(&threads[i], NULL, averageSubThread, threadData[i]);
   current += rowsNow;
}
```

The average thread uses the same row division idea as multiplication. The rows are divided between subthreads, and each subthread calculates the sum of its assigned rows first.

```
int sum = 0;
for(int i = current; i < current + rowsNow; i++){
   for(int j = 0; j < cols; j++){
      sum += avgMatrix[i][j];
   }
}
pthread_mutex_lock(&lock);
totalSum += sum;
pthread_mutex_unlock(&lock);
```

After calculating the partial sum, each subthread adds it to totalSum. Since totalSum is shared between all the average subthreads, more than one subthread may try to change it at the same time. Because of that, I used pthread_mutex_lock() before changing totalSum and pthread_mutex_unlock() after finishing. This allows one subthread at a time to change totalSum. I used pthread_join() to wait until all the average subthreads finish, then I divide totalSum by rows * cols to get the final average.

4.5 Race Condition in My Implementation

The race condition I found is in the average subthreads. Every subthread calculates its own sum, but after that all of them need to add their result to the same totalSum.

To show the race condition, I changed srand(time(NULL)) to srand(1) and I used fixed dimensions for the matrices. I did this so that the same matrix is generated in both runs before and after using the mutex.

```
int sum = 0;
for(int i = current; i < current + rowsNow; i++){
   for(int j = 0; j < cols; j++){
      sum += avgMatrix[i][j];
   }
}
totalSum += sum;
```

sum is separate for every subthread, but totalSum is shared. If more than one subthread reaches totalSum += sum at the same time, they may read the same old value before the other update is completed. This means one of the additions can be lost and the final average can be wrong.

I solved this by adding the mutex only around the part that changes totalSum.

```
int sum = 0;
for(int i = current; i < current + rowsNow; i++){
   for(int j = 0; j < cols; j++){
      sum += avgMatrix[i][j];
   }
}
pthread_mutex_lock(&lock);
totalSum += sum;
pthread_mutex_unlock(&lock);
```

After the subthread calculates its own sum, pthread_mutex_lock() is called before changing totalSum. If another subthread is already changing it, the other subthreads wait. After the addition, pthread_mutex_unlock() releases the mutex and another subthread can enter this part.

A point worth mentioning is that I checked the race condition in two levels. The first level is between the three main threads. Multiplication, transpose and average all use Matrix A and Matrix B, but they only read their values. Also, multiplication stores its result in multRes, transpose stores its result in transRes and average stores its result in avgResult. So, I did not need a mutex between the three main threads.

The second level is between the subthreads. Every multiplication subthread calculates different rows inside multRes, so the subthreads do not change the same rows and I did

not need a mutex between them. The average is different because all the average subthreads need to change the same totalSum, so this is where I used the mutex.

After creating the main threads, I used pthread_join() for them, and the same thing is done for the multiplication and average subthreads. This makes the thread that created them wait until they finish before it continues.

5. Client-Server Implementation

For the client-server implementation, I kept the same matrix scenario and the same multithreaded approach that I used in Q6, but I divided the work between a client and a server. The client generates Matrix A and Matrix B, asks the user which matrix to transpose and which matrix to calculate the average for, then sends the matrices and the choices to the server. The server receives the data, performs the same multiplication, transpose and average operations from Q6 using threads, then sends the results back to the same client.

```
write(sockfd, &rA, sizeof(int));
write(sockfd, &cA, sizeof(int));
write(sockfd, &rB, sizeof(int));
write(sockfd, &cB, sizeof(int));
for(int i = 0; i < rA; i++){
   write(sockfd, matrixA[i], cA * sizeof(int));
}
for(int i = 0; i < rB; i++){
   write(sockfd, matrixB[i], cB * sizeof(int));
}
write(sockfd, &choiceOne, sizeof(char));
write(sockfd, &choiceTwo, sizeof(char));
```

After generating Matrix A and Matrix B and taking the choices from the user, I send the dimensions first, then Matrix A and Matrix B row by row. After that, I send choiceOne and choiceTwo so that the server knows which matrix should be transposed and which matrix should be used to calculate the average.

```
while(1){
  clilen = sizeof(cli_addr);
  int *newsockfd = malloc(sizeof(int));
  *newsockfd = accept(sockfd, (struct sockaddr *) &cli_addr, &clilen);
  pthread_create(&tid, NULL, clientThread, newsockfd);
  pthread_detach(tid);
}
```

I used a while loop so that the server can continue accepting clients instead of stopping after one client. When a client connects, I create a separate thread for that client and pass its socket to the thread. The server can then go back and accept another client while the

previous client is still being handled. Therefore, more than one client can be handled concurrently.

```
pthread_mutex_lock(&clientLock);
int client = clientCount;
clientCount++;
pthread_mutex_unlock(&clientLock);
```

Since more than one client can be working with the server at the same time, I used the client number with the matrices, dimensions, choices and results. For example, position 0 is used for the first client and position 1 is used for the second client. This keeps the data of each client separate while their threads are running concurrently. I used clientCount to give every client a different number. Since more than one client thread could reach this part at the same time, I used a mutex while reading and increasing clientCount.

```
pthread_create(&firstThread, NULL, multiplicationThread, &client);
pthread_create(&secondThread, NULL, transposeThread, &client);
pthread_create(&thirdThread, NULL, averageThread, &client);
pthread_join(firstThread, NULL);
pthread_join(secondThread, NULL);
pthread_join(thirdThread, NULL);
```

For the calculations, I used the same multithreaded approach that I used in Q6. The multiplication, transpose and average logic is still the same, including the segmentation used for multiplication and average. The difference in this implementation is that I also pass the client number to the operation threads and subthreads because calculations for more than one client can be running at the same time.

```
threadData[i][0] = client;
threadData[i][1] = current;
threadData[i][2] = rowsNow;
```

In Q6, the subthreads only needed the starting row and the number of rows assigned to them. Here, I also include client so that each subthread knows which client's matrices and results it should work with. The rest of the calculation logic remains the same as Q6.

```
for(int i = 0; i < rA[client]; i++){
  write(newsockfd, multRes[client][i], cB[client] * sizeof(int));
}
write(newsockfd, &avgResult[client], sizeof(double));
```

After the multiplication, transpose and average threads finish, I send the multiplication result first, followed by the transpose result and then the average result. These results are sent using the socket of the same client that sent the matrices. On the client side, I receive the results in the same order and print them

```
pthread_mutex_lock(&logLock);
logFile = open("serverLog.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
```

```
...
close(logFile);
pthread_mutex_unlock(&logLock);
```

I also added a log file on the server side to store the transactions for every client. The log contains when the client connected, Matrix A, Matrix B, the multiplication result, transpose result, average result and when the client disconnects. I used O_APPEND so that the new information is added without removing the transactions that were already stored. Since the server can handle more than one client concurrently, more than one client thread could try to write to the log file at the same time. Therefore, I used logLock so that one client thread writes to the file at a time and the information from different clients does not get mixed together.

6. C vs Shell Scripting

C programming and shell scripting can both be used to perform system services, but they work in different ways. C is compiler based, where the source code is compiled into an executable before it is executed. Shell scripting is interpreter based, where Bash reads the instructions from the source file and executes each instruction. Programming languages are generally more powerful and faster than scripting languages, while shell scripting is useful to automate tasks that would otherwise require many commands. In C, I can use the related system calls directly inside the program, while in shell scripting I can perform the same task using commands interpreted by Bash.

Criteria	C	Shell Scripting
Execution	The source code is compiled into an executable before it is executed.	The source code is not compiled into an executable, as Bash reads and executes each instruction using the interpreter.
Speed	C is generally more powerful and faster because the program is compiled before it is executed.	Shell scripting is generally slower because the instructions are read and executed by the interpreter.
Automation	In C, I need to write the required operations inside a program and compile the program before executing it.	Shell scripting can be used to automate a number of commands that would otherwise need to be entered manually.
Creating Programs	A C program is written as source code and then compiled to create the executable file.	A shell script is written as a text file, and #!/bin/bash is used to tell Linux to use the Bash interpreter to execute it.

Running Commands or Processes	In C, I can use fork() to create a child process and then use exec() to change the child into the required program.	In shell scripting, I can write the required command directly inside the script and Bash executes it.
Communication Channel	In C, I can create a pipe using pipe(), or use a named pipe, then use read() and write() to transfer the data between processes.	A named pipe can be created using mkfifo, and it can be accessed using commands such as cat.
Files	In C, I can use open(), read(), write() and close() to work with the file and control the data that is being read or written.	In shell scripting, I can use commands such as cat and cp to perform the file operation directly.

6.1 Code Examples

Copying Files

Shell:

```
cp file1.txt file2.txt
```

C:

```
input = open("file1.txt", O_RDONLY);
output = open("file2.txt", O_WRONLY | O_CREAT, 0644);
while((n = read(input, buffer, 100)) > 0){
    write(output, buffer, n);
}
close(input);
close(output);
```

In shell scripting, I can copy the file directly using cp. In C, I have to open the first file and the second file, then read from the first one and write the data into the second one. So, shell scripting is easier for this operation, while in C I can control the reading and writing.

Create a file & Change Permission

Shell:

```
touch results.txt
chmod 444 results.txt
```

C:

```
FILE *f = fopen("results.txt", "w");
fclose(f);
chmod("results.txt", 0444);
```

In shell scripting, I can create the file using touch and change its permission using chmod. In C, I create the file using open(), close it, then change its permission using chmod(). So, shell scripting needs less code to do the same operation.

Reading & Displaying Files

Shell:

```
cat results.txt
```

C:

```
fd = open("results.txt", O_RDONLY);
while((n = read(fd, buffer, 100)) > 0){
    printf("%s", n, buffer);
}
close(fd);
```

In shell scripting, I just use cat to read the file. In C, I open the file first, read from it, print what was read, then close it. So, the shell script takes less steps here, while in C I can choose how much data I read.

7 Implementing Shell Scripting

I used a shell script to automate the client-server scenario. Instead of running the server and then opening the client 50 times, the script will do these steps automatically. I also used some commands for the files and permissions, then grep to search inside the log file.

```
rm -rf clientOutputs
mkdir clientOutputs
touch serverLog.txt
echo -n "" > serverLog.txt
```

First, I remove the old clientOutputs directory and create it again. I also create serverLog.txt if it does not exist and clear the old data from it. So, every time I run the script I will have the new outputs only.

```
echo "Compiling..."
gcc server.c -o server -pthread
gcc client.c -o client
```

Then I compile the server and client from the script instead of compiling them manually before every run. I used -pthread with the server because the server uses threads.

```
chmod 700 server
chmod 700 client
chmod 600 serverLog.txt
```

After that, I change the permissions. I used 700 for the server and client so the owner can execute them, and 600 for serverLog.txt so the owner can read and write the file.

```
echo "Starting Server"
./server > serverOutput.txt &
sleep 1
```

Then I start the server in the background using &, so the script can continue to the next commands. I used sleep 1 after it so the server starts before the clients.

```
echo "Starting 50 Clients"
for (( i = 1 ; i <= 50; i++ ))
do
 ./client > clientOutputs/client_$i.txt &
done
```

I used a for loop to run the client 50 times instead of running every client by myself. I also used & so the clients can run at the same time. Every client output will be saved in a different file.

Also, The client does not ask me to choose Matrix A or B anymore because I changed the two choices to be random. So, the clients can complete the scenario without waiting for input from me.

After starting the clients, I used sleep 10 before checking the log file to give the clients time to finish their work.

```
echo "Connected Clients:"
grep -cE "Client [0-9]+ Connected" serverLog.txt
```

I used grep to count the clients that connected to the server. -c counts the lines, while [0-9]+ is for the client number.

```
echo "Disconnected Clients:"
grep -cE "Client [0-9]+ Disconnected" serverLog.txt
```

I used the same idea to count the clients that disconnected from the server.

```
echo "Client Records:"
grep -nE "Client [0-9]+ (Connected|Disconnected)" serverLog.txt
```

Here I search for the connected and disconnected clients together.
(Connected|Disconnected) means that it can find any one of the two words, and -n shows the line number.

```
echo "Calculation Results:"
grep -nE "(Multiplication Result|Transpose Of Matrix [AB]|Average Of
Matrix [AB])" serverLog.txt
```

I also search inside the log for the multiplication, transpose and average results. [AB] means that the matrix can be A or B.

7.1 Full Script

```
#!/bin/bash
rm -rf clientOutputs
mkdir clientOutputs
touch serverLog.txt
echo -n "" > serverLog.txt
echo "Compiling..."
gcc server.c -o server -pthread
gcc client.c -o client
chmod 700 server
chmod 700 client
chmod 600 serverLog.txt
echo "Starting Server"
./server > serverOutput.txt &
sleep 1
echo "Starting 50 Clients"
for (( i = 1 ; i <= 50; i++ ))
do
 ./client > clientOutputs/client_$i.txt &
done
sleep 10
echo "Connected Clients:"
grep -cE "Client [0-9]+ Connected" serverLog.txt
echo "Disconnected Clients:"
grep -cE "Client [0-9]+ Disconnected" serverLog.txt
echo "Client Records:"
grep -nE "Client [0-9]+ (Connected|Disconnected)" serverLog.txt
echo "Calculation Results:"
grep -nE "(Multiplication Result|Transpose Of Matrix [AB]|Average Of
Matrix [AB])" serverLog.txt
echo "Automation Finished"
```

References

1. https://www.geeksforgeeks.org/operating-systems/system-programs-in-operating-system/
2. System Calls, Signals, & Interrupts - CS 3410 Fall 2025
3. Difference between Multiprocessing and Multithreading - GeeksforGeeks
4. Matrix Multiplication in C - GeeksforGeeks
5. How to dynamically allocate a 2D array in C? - GeeksforGeeks
6. Transpose of a Matrix in C - GeeksforGeeks
7. Measure execution time with high precision in C/C++ - GeeksforGeeks
8. How to insert CODE snippet in MS Word document like a pro
9. Semaphores in Process Synchronization - GeeksforGeeks

10. Thread Pool in C++ - GeeksforGeeks
11. Difference between Semaphore and Condition Variable - GeeksforGeeks